

HEART DISEASE RISK CLASSIFICATION

XGBoost Machine Learning Implementation & Streamlit Application

Academic Project Documentation

Based directly on the submitted Google Colab notebook and implemented application

Problem Domain	Heart Disease Risk Classification
Algorithm	XGBoost Classifier
Dataset After Duplicate Removal	5,510 records × 19 columns
Train / Test Split	4,408 / 1,102 samples

1. Project Overview

The notebook implements an educational binary classification workflow for heart disease risk. The target variable is `heart_disease`, with 0 representing the negative class and 1 representing the positive class. The workflow covers data loading, dataset inspection, duplicate removal, feature-target separation, stratified train-test splitting, XGBoost model definition and training, prediction, evaluation, confusion-matrix analysis, ROC-curve visualization, feature importance, and serialization of the trained model for use in the Streamlit application.

The accompanying Streamlit application provides a user interface for entering the same 18 model features and passing them to the serialized XGBoost model for classification.

2. Libraries and Technologies

The notebook imports the following libraries and components:

- Pandas — loading and manipulating tabular data.
- NumPy — numerical operations.
- Matplotlib — plotting and visualization.
- Seaborn — statistical visualization, including the confusion matrix heatmap.
- scikit-learn — train-test splitting, classification metrics, confusion matrix, and ROC visualization.
- XGBoost — implementation of the `XGBClassifier` used for the main machine-learning model.
- Joblib — serialization of the trained model and feature-name list for deployment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay, RocCurveDisplay

from xgboost import XGBClassifier
```

3. Dataset Loading

The dataset is loaded into a Pandas `DataFrame` using `read_csv()`. The notebook uses the file `heart_disease_prediction_dataset.csv`.

```
data = pd.read_csv("heart_disease_prediction_dataset.csv")
```

4. Dataset Understanding and Inspection

The notebook performs several basic data-understanding operations before model training. These operations establish the structure, variables, data types, and descriptive statistics of the dataset.

- `data.columns` — lists all dataset attributes.
- `data.head()` — displays the first five records.
- `data.info()` — reports row count, column count, non-null counts, data types, and memory usage.

- `data.describe()` — provides descriptive statistics such as count, mean, standard deviation, quartiles, minimum, and maximum.
- `data.nunique()` — checks the number of unique values in every column.

```
print(data.columns)
print("\n" + "=" * 128)
print("DATASET OVERVIEW - FIRST 5 ROWS".center(128))
print("=" * 128)
print(data.head())
print("\n" + "=" * 128)
print("Dataset Structure".center(128))
print("=" * 128)
data.info()
print("\n" + "=" * 128)
print("STATISTICAL SUMMARY".center(128))
print("=" * 128)
print(data.describe())
```

Notebook result: the original dataset contains 5,568 rows and 19 columns. All 19 columns have 5,568 non-null observations. The data types are 5 float64 columns and 14 int64 columns.

The target `heart_disease` has a mean of approximately 0.094648, indicating that the positive class is much less frequent than the negative class in the original dataset.

5. Duplicate Detection and Removal

Duplicate records are checked using `duplicated()`. The notebook then removes duplicate rows using `drop_duplicates()`. This is a data-cleaning step intended to prevent repeated records from unnecessarily influencing model training and evaluation.

```
print("Duplicates before:", data.duplicated().sum())

data = data.drop_duplicates()

print("Duplicates after:", data.duplicated().sum())
print("New shape:", data.shape)
```

Observed result: 58 duplicate rows were present initially. After removal, zero duplicates remained, and the dataset size became 5,510 rows × 19 columns.

6. Feature–Target Separation

The target variable `heart_disease` is separated from the predictor variables. `X` contains the 18 input features, while `y` contains the target labels.

```
X = data.drop("heart_disease", axis=1)
y = data["heart_disease"]

print("Features : ", X.shape)
print("Target : ", y.shape)
```

The resulting dimensions are:

- `X`: 5,510 rows × 18 features

- y: 5,510 target values

6.1 Input Features

- taking_bp_meds
- taking_cholesterol_meds
- age
- sex
- race_ethnicity
- education
- poverty_income_ratio
- taking_insulin
- taking_diabetes_pills
- told_prediabetes
- told_stroke
- family_history_heart_attack
- diabetes
- smoking_ever
- smoking_current
- physically_active
- hypertension
- high_cholesterol

7. Train-Test Split

The dataset is divided into training and testing subsets using `train_test_split()`. The notebook uses a `test_size` of 0.2, meaning 20% of the cleaned data is reserved for testing. `stratify=y` is used to preserve the class distribution between the training and testing subsets.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)
```

Observed result: 4,408 samples are used for training and 1,102 samples are used for testing.

Implementation note: the train-test split does not specify `random_state`, so the exact split can change between notebook executions even though the XGBoost model itself specifies `random_state=42`.

8. XGBoost Model Definition

The project uses `XGBClassifier` from XGBoost. XGBoost is a gradient-boosting method that constructs an ensemble of decision trees sequentially, with later trees learning from errors or residual patterns identified by earlier trees.

```
model = XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.05, random_state=42)
```

8.1 Implemented Hyperparameters

Parameter	Value	Role
n_estimators	200	Number of boosting trees

max_depth	4	Maximum depth of each tree
learning_rate	0.05	Step size used during boosting
random_state	42	Controls model-level randomness

9. Model Training

The model is trained using the training features and training labels. During this step, XGBoost learns relationships between the 18 input variables and the binary heart_disease target.

```
model.fit(X_train, y_train)
```

10. Prediction

After training, the model generates class predictions for the unseen test set using model.predict(). Each test sample receives a predicted class label of 0 or 1.

```
y_pred = model.predict(X_test)
```

11. Model Evaluation

The notebook evaluates the classifier using accuracy, precision, recall, F1 score, and ROC-AUC. These metrics provide different views of classification performance.

```
accuracy = accuracy_score(y_test,
y_pred)
precision = precision_score(y_test,
y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("ROC AUC Score:", roc_auc)
```

Metric	Observed value	Meaning
Accuracy	0.89837	Proportion of all test predictions classified correctly.
Precision	0.38710	Among samples predicted positive, proportion that are actually positive.

Recall	0.11429	Among actual positive samples, proportion correctly identified.
F1 Score	0.17647	Harmonic mean of precision and recall.
ROC-AUC as implemented	0.54761	Calculated in the notebook using <code>y_test</code> and <code>y_pred</code> .

The classification report gives the class-specific results shown below:

```
print(classification_report(y_test, y_pred))
```

12. Classification Report Interpretation

The test set contains 997 samples in class 0 and 105 samples in class 1. The model performs much better on class 0 than on class 1. In particular, the recall for class 1 is 0.11, meaning that the implemented classifier identifies only a relatively small fraction of the positive test cases.

- Class 0: precision 0.91, recall 0.98, F1-score 0.95.
- Class 1: precision 0.39, recall 0.11, F1-score 0.18.
- Overall accuracy: approximately 0.90.
- Macro-average F1: 0.56, which treats both classes equally.
- Weighted-average F1: 0.87, which is influenced by the larger class-0 support.

13. Confusion Matrix

A confusion matrix compares actual labels with predicted labels. The notebook creates a 2×2 matrix and visualizes it using a Seaborn heatmap.

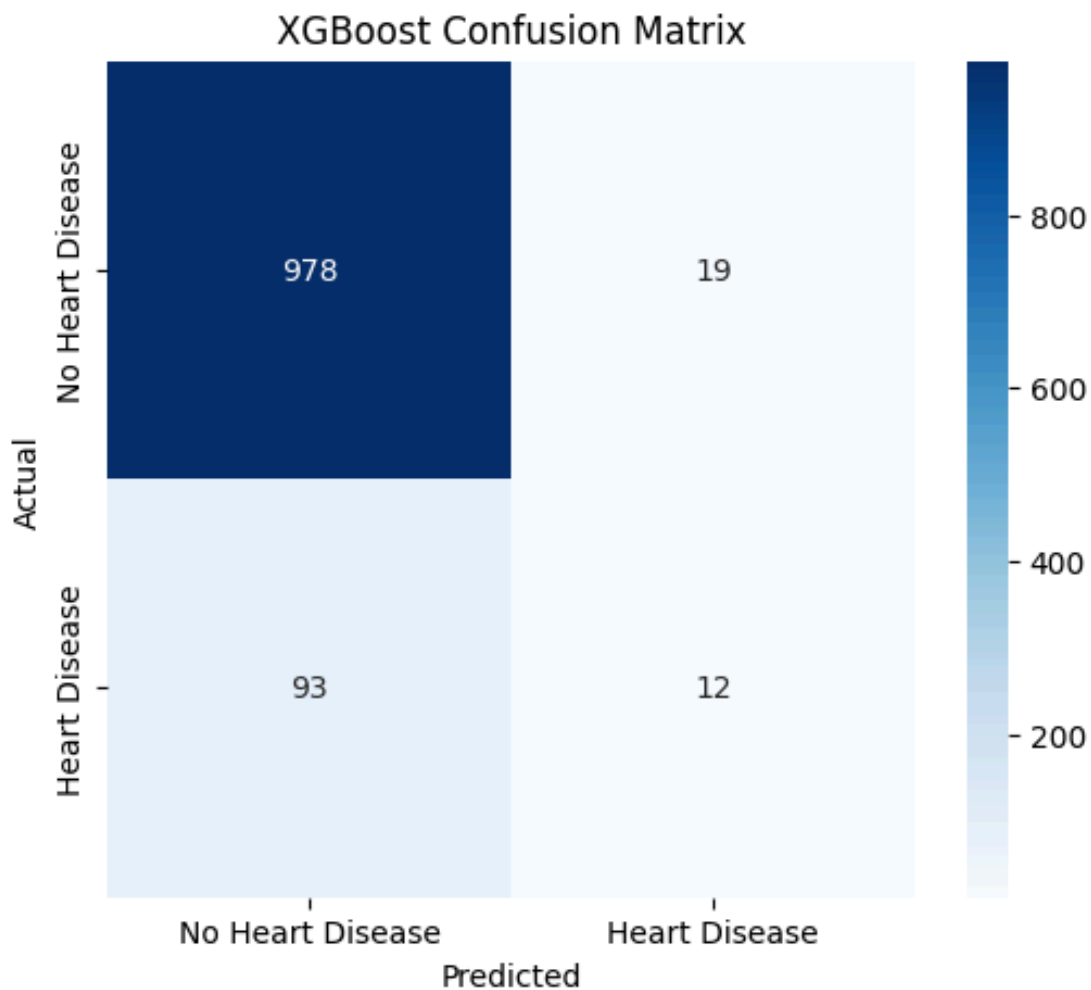
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Heart Disease", "Heart Disease"],
    yticklabels=["No Heart Disease", "Heart Disease"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("XGBoost Confusion Matrix")

plt.show()
```

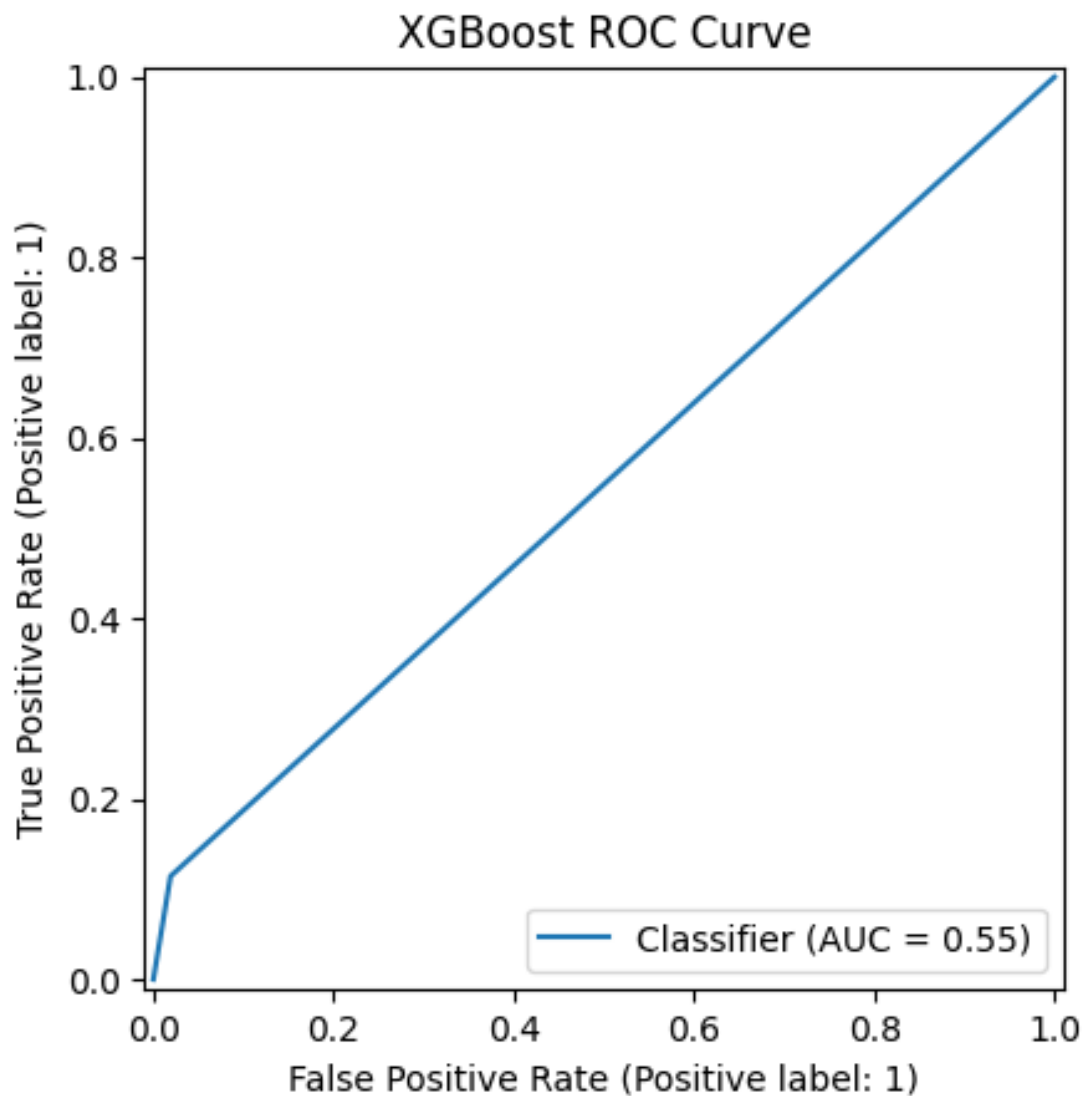


The displayed matrix contains: True Negative = 978, False Positive = 19, False Negative = 93, and True Positive = 12. The relatively high number of false negatives is consistent with the low class-1 recall reported above.

14. ROC Curve

The notebook uses `RocCurveDisplay.from_predictions()` to visualize the classifier's ROC curve.

```
RocCurveDisplay.from_predictions(  
    y_test,  
    y_pred  
)  
  
plt.title("XGBoost ROC Curve")  
plt.show()
```



The notebook displays an AUC of approximately 0.55. This is the value produced by the implementation.

Implementation note: the notebook passes `y_pred` (hard 0/1 predictions) to both `roc_auc_score()` and `RocCurveDisplay.from_predictions()`. A conventional ROC-AUC analysis normally uses continuous scores such as `model.predict_proba(X_test)[:, 1]`. Therefore, the reported 0.54761 should be described as the ROC-AUC value calculated by this notebook implementation, rather than as a probability-score ROC-AUC.

15. XGBoost Feature Importance

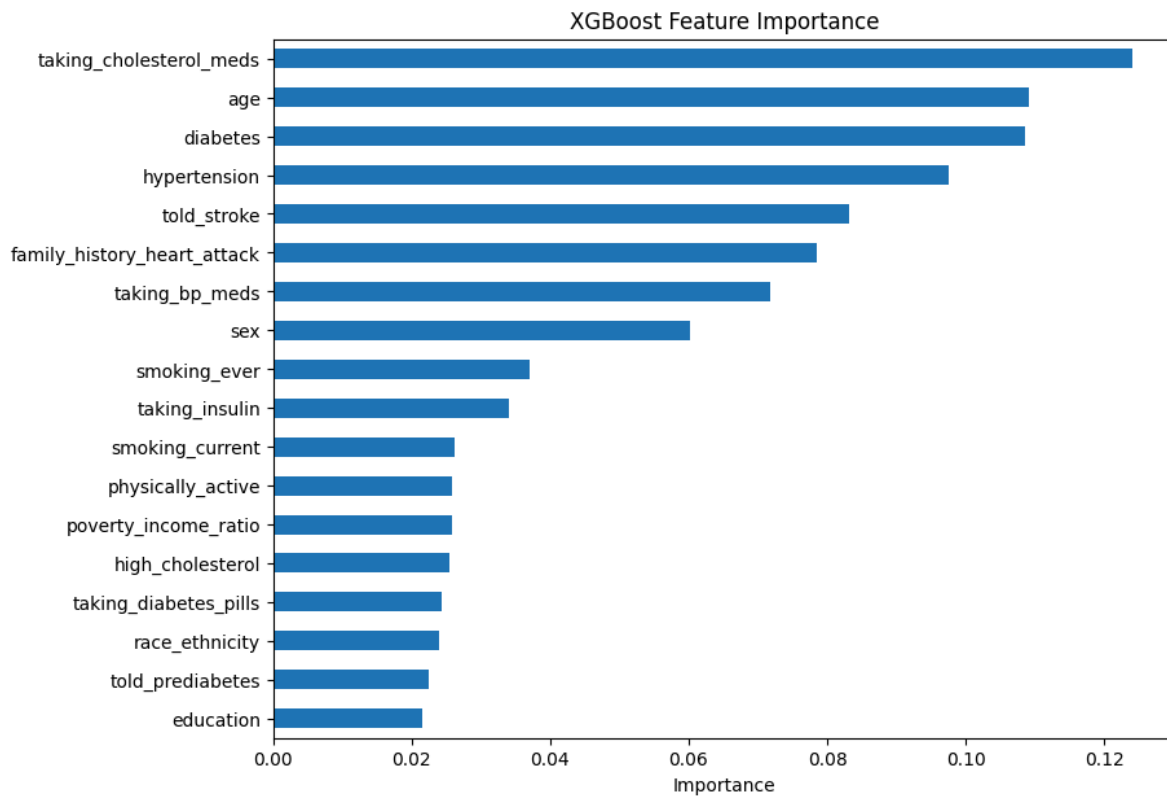
The notebook obtains `model.feature_importances_` and maps the values to the original feature names. The resulting values are sorted and displayed as a horizontal bar chart.

```
importance = pd.Series(  
    model.feature_importances_,  
    index=X.columns  
)  
.sort_values(ascending=True)  
  
plt.figure(figsize=(9,7))
```

```
importance.plot(kind="barh")

plt.title("XGBoost Feature Importance")
plt.xlabel("Importance")

plt.show()
```



The visualization shows the relative contribution of the input features to the trained XGBoost model according to XGBoost's feature-importance measure. This describes model behavior; it does not establish that a feature causes heart disease.

16. Model Serialization for Deployment

After evaluation, the trained XGBoost model is saved using Joblib. This allows the Streamlit application to load the already-trained model instead of retraining it every time the application starts.

```
import joblib

# Save model
joblib.dump(model, "xgboost_heart_disease_model.pkl")

print("Model saved successfully!")
```

The notebook also stores the exact feature-name ordering used by X:

```
feature_names = X.columns.tolist()

joblib.dump(feature_names, "feature_names.pkl")
```

```
print(feature_names)
```

Storing the feature names is important for deployment because the application must construct its input DataFrame in the same feature order expected by the trained model.

17. Streamlit Application Implementation

The deployed application provides a user-facing form corresponding to the 18 model inputs. The interface groups the inputs into health information and medical/lifestyle information and uses appropriate Streamlit widgets such as number inputs and Yes/No selection controls.

- Loads `xgboost_heart_disease_model.pkl` using `joblib`.
- Loads `feature_names.pkl` to preserve the training feature order.
- Collects all 18 input values from the user.
- Builds a one-row Pandas DataFrame from those values.
- Reorders the columns according to the stored `feature_names` list.
- Calls `model.predict()` to obtain the binary classification.
- Uses `model.predict_proba()` to display the model's class-1 probability.
- Displays an educational disclaimer stating that the output is not a medical diagnosis.

17.1 Application Flow

User Input → Pandas DataFrame → Feature Ordering → Serialized XGBoost Model → Class Prediction + Class-1 Probability → Streamlit Result

18. Educational and Technical Limitations

- The current test-set results show substantially weaker performance for class 1 than class 0.
- The dataset is imbalanced, so accuracy alone does not adequately describe positive-class detection.
- The notebook's ROC-AUC implementation uses hard predictions rather than continuous predicted probabilities.
- The train-test split does not specify `random_state`, so the exact train/test partition is not fully reproducible.
- Feature importance should not be interpreted as causal evidence.
- The model is an educational machine-learning implementation and is not clinically validated.
- The Streamlit prediction must not be interpreted as a medical diagnosis or clinical recommendation.

19. End-to-End Implementation Summary

1. Import the Python, visualization, scikit-learn, XGBoost, and Joblib libraries.
2. Load the heart disease CSV dataset.
3. Inspect columns, sample records, structure, descriptive statistics, and unique values.
4. Detect and remove 58 duplicate rows, producing 5,510 clean records.
5. Separate `heart_disease` as the target and retain 18 predictor variables.
6. Create a stratified 80/20 train-test split: 4,408 training and 1,102 testing records.
7. Define an `XGBClassifier` with 200 estimators, maximum depth 4, learning rate 0.05, and `random_state 42`.
8. Train the model on the training subset.

9. Generate predictions on the test subset.
10. Evaluate using accuracy, precision, recall, F1 score, classification report, confusion matrix, and ROC visualization.
11. Inspect XGBoost feature importance.
12. Serialize the trained model and feature-name order using Joblib.
13. Use the serialized artifacts in the Streamlit application for interactive classification.

20. Conclusion

The notebook implements a complete educational machine-learning classification pipeline from raw tabular data inspection through model training, evaluation, interpretation, and deployment preparation. XGBoost is used as the primary classifier, while scikit-learn supplies the data split and evaluation utilities. The resulting model achieves approximately 89.84% accuracy on the recorded test split, but its class-1 recall is approximately 11.43%, demonstrating why multiple evaluation metrics are necessary for this imbalanced classification task.

The serialized model and feature ordering form the bridge between the Colab training workflow and the Streamlit application, enabling the trained model to be used interactively without retraining.

21. Academic / Medical Disclaimer

This project is intended strictly for educational and academic purposes. The model output represents patterns learned from the selected dataset and must not be interpreted as a medical diagnosis, clinical recommendation, treatment decision, or substitute for professional medical advice.